

DERİN KAZI

Çok Oyunculu Ekip Kazısı Modu

1'den 10 Oyuncuya Kadar Ölçeklenen Kooperatif Kazı Sistemi

Mevcut Flutter kod tabanı incelemesi · Oyun tasarımı · Teknik mimari · Yol haritası

Doküman Türü

Uzman seviyesi tasarım + mimari doküman (kod önerisi içermez)

Kapsam

Derin Kazı — mevcut kaynak kodu analizi ve çok oyunculu genişletme tasarımı

Tarih

22 Ağustos 2026

İÇİNDEKİLER

0. Yönetici Özeti

1. Mevcut Kod Tabanı Analizi

- 1.1 Genel Mimari
- 1.2 Tespit Edilen Kilit Noktalar

2. Konsept ve Hedefler

- 2.1 Fikir: "Ekip Kazısı" Modu
- 2.2 Neden Bu Tasarım?
- 2.3 Kapsam Dışı Bırakılanlar (Bilinçli Sınırlar)

3. Oyun Tasarımı — Ekip Kazısı Modu

- 3.1 Oyun Modları
- 3.2 Oda / Lobi Sistemi
- 3.3 Paylaşımlı Harita ve Ortak Hasar Havuzu
- 3.4 Ekonomi: Kişisel Kazanç + Takım Bonusu
- 3.5 Enerji ve İşbirliği Mekanikleri
- 3.6 Sosyal Katman
- 3.7 Oturum Yaşam Döngüsü
- 3.8 Bağlantı Kopması, Ayrılma ve Host Devri
- 3.9 Oyuncu Sayısına Göre Zorluk Ölçekleme

4. Kullanıcı Akışı ve Yeni Ekranlar

5. Teknik Mimari

- 5.1 Backend Seçimi
- 5.2 Veri Modeli (Realtime Database Şeması)
- 5.3 Gerçek Zamanlı Senkronizasyon Akışı
- 5.4 Flutter Tarafında Mimari Genişleme
- 5.5 Yeni ve Değiştirilecek Dosyalar (Plan — Kod Değil)
- 5.6 Ekranlar Arası Durum Yönetimi
- 5.7 Güvenlik Kuralları ve Oda Temizliği
- 5.8 Görsel/UI Değişiklikleri Özeti

6. Maliyet ve Ölçeklenebilirlik Tahmini

7. Hile Önleme ve Adil Oyun

8. Monetizasyon Fırsatları

9. Uygulama Yol Haritası

- Faz 1 — Temel (MVP): Davetli Ekip Kazısı
- Faz 2 — Genişletme
- Faz 3 — Olgunlaşma (talebe bağlı)

10. Riskler ve Azaltım Stratejileri

11. Sonuç

0. Yönetici Özeti

Bu doküman, Derin Kazı adlı 2B kazı/madencilik bulmaca oyununun mevcut Flutter kod tabanının uçtan uca incelemesini ve oyuna çok oyunculu (multiplayer) Ekip Kazısı modu eklenmesi için uzman seviyesinde bir oyun tasarımı ile teknik mimari önerisini içerir. Talep açık: oyun şu anda tamamen tek kişilik (yerel, çevrimdışı) çalışıyor; hedef, aynı oyunun tek kişi, 2 kişi, 3 kişi... 10 kişiye kadar oynanabilmesini ve oyuncu sayısının oda kurucusu (host) tarafından serbestçe, 10 kişilik bir tavana kadar seçilebilmesini sağlamak.

Bu, sadece "birden fazla imleç aynı ekranda" gibi kozmetik bir ekleme değil; ekonomi, adil ödül paylaşımı, gerçek zamanlı senkronizasyon, hile önleme ve bağlantı kopması gibi çok oyunculu oyun geliştirmenin klasik problemlerini çözmeyi gerektiren orta ölçekli bir mimari genişlemedir. Doküman bu problemleri tek tek ele alır ve mevcut kod yapısıyla uyumlu, aşamalı olarak uygulanabilir bir çözüm sunar.

Bu dokümanda ne var

Mevcut kodun (15 dosya, ~2.240 satır) mimari analizi ve çok oyunculuya hazır olma değerlendirmesi.

"Ekip Kazısı" modu için uçtan uca oyun tasarımı: lobi/oda sistemi, paylaşımlı harita, ödül dağıtımı, sosyal mekanikler.

Backend seçimi karşılaştırması ve önerilen mimari (Firebase Realtime Database + Cloud Functions).

Yeni veri modelleri, klasör yapısı ve mevcut dosyalarda değişecek noktaların listesi (kod yazılmadan, sadece plan).

Maliyet tahmini, riskler, hile önleme stratejisi ve 3 fazlık uygulama yol haritası.

Not: Talebiniz doğrultusunda bu doküman tasarım ve mimari düzeyinde hazırlanmıştır; içinde üretim kodu bulunmaz. Onayınızın ardından, burada tanımlanan mimariye göre kod geliştirme aşamasına geçilebilir.

1. Mevcut Kod Tabanı Analizi

Yüklediğiniz kaynak kodu (Flutter, Riverpod tabanlı) satır satır incelendi. Aşağıda mimarinin özeti, güçlü yönleri ve çok oyunculu geçiş için önemli olan tespitler yer alıyor.

1.1 Genel Mimari

Proje, özellik bazlı (feature-first) katmanlı bir mimari izliyor: *domain* (modeller ve servisler), *application* (durum yönetimi / iş mantığı) ve *presentation* (ekranlar ve widget'lar) net şekilde ayrılmış. Durum yönetimi flutter_riverpod ile *StateNotifier* deseni kullanılarak yapılmış. Bu, hem okunabilir hem de genişletmeye uygun, sağlam bir temel — çok oyunculu katman bu yapıyı bozmadan üzerine eklenebilir.

Dosya	Satır	Sorumluluk
game_notifier.dart	325	Tüm oyun mantığının kalbi: kazı, hasar, ödül, aşama ilerletme, yükseltme satın alma, prestij sıfırlama, enerji zamanlayıcısı.
grid_generator.dart	111	13×23'lük ızgarayı rastgele (seed'siz) üretir; taş tipi olasılıkları ve HP/ödül formülleri burada.
grid_model.dart / tile_model.dart	66 / 84	Izgara ve tek kare (tile) veri modelleri. tile_model.dart zaten toJson/fromJson içeriyor (serileştirmeye hazır).
player_state_model.dart	122	Oyuncunun altın, gem, enerji, rütbe, yükseltmeleri — tamamen yerel/tekil oyuncuya özel.
upgrade_model.dart	84	Kazma/Çekiç/Şans/Enerji yükseltme tanımları; JSON destekli.
hud_bar.dart / shop_panel.dart / mining_grid_view.dart / directional_pad.dart / bottom_progress_bar.dart	367 / 333 / 64 / 109 / 78	Sunum katmanı: üst bilgi çubuğu, mağaza paneli, ızgara görünümü, yön tuşları, enerji çubuğu.
tile_gridPainter.dart	289	CustomPainter ile ızgaranın ve tek oyuncu imlecinin çizimi.

1.2 Tespit Edilen Kilit Noktalar

Çok oyunculu tasarımı doğrudan etkileyen beş teknik gerçek var:

- 1 Oyun tamamen istemci tarafı (client-side) ve tek örnekli. *GameNotifier* hem durumu tutuyor hem kuralları uyguluyor; sunucu, ağ katmanı veya kimlik doğrulama yok. Bu normal — mevcut ürün çevrimdışı bir tek oyunculu deneyim olarak tasarlanmış.
- 2 Izgara üretimi seed kullanmıyor (*dart:math Random()* — cihaza özel, tekrarlanamaz). Aynı haritayı birden fazla cihazda aynı şekilde üretmek için bu, deterministik (seed'li) bir üretici ile değiştirilmeli.
- 3 Tek bir *Position playerPosition* alanı ızgara modelinde tanımlı — yapı, doğası gereği "tek oyuncu" varsayımıyla kurulmuş. Çoklu oyuncu pozisyonları için genişletilmesi gerekiyor.
- 4 Ödül/hasar hesaplaması tamamen istemci tarafında ve doğrudan uygulanıyor (*moveOrDig* içinde anında altın/gem veriliyor). Çevrimiçi çok oyunculuda bu yaklaşım hileye açık olur; sunucu-yetkili (server-authoritative) doğrulamaya taşınmalı.
- 5 Modeller zaten JSON serileştirmeye kısmen hazır (*TileModel*, *UpgradeModel*). Bu, ağ üzerinden veri taşımayı kolaylaştıran olumlu bir başlangıç noktası.

- 6 UI bileşenleri (D-Pad, ızgara görünümü, HUD) mantıktan iyi ayrılmış. Bu sayede yeni bir "çok oyunculu veri kaynağı" bağlansa bile mevcut görsel bileşenlerin büyük bölümü neredeyse değişmeden yeniden kullanılabilir — bu, projenin en değerli avantajı.

Değerlendirme

Özetle: kod kalitesi ve katmanlı yapı, çok oyunculu bir genişleme için sağlam bir zemin. Eksik olan şey "kötü kod" değil, hiç var olmayan bir ağ / senkronizasyon katmanı. Aşağıdaki tasarım, bu katmanı mevcut yapıyı kırmadan, *Repository deseni* ile ekleyecek şekilde kurgulanmıştır (bkz. Bölüm 6.4).

2. Konsept ve Hedefler

2.1 Fikir: "Ekip Kazısı" Modu

Derin Kazı'nın mevcut tek oyunculu deneyimi ("Solo Kazı") olduğu gibi korunur ve varsayılan mod olmaya devam eder. Buna ek olarak, ana menüye "Ekip Kazısı" adlı yeni bir mod eklenir: bir oyuncu (host) bir oda kurar, katılacak en fazla oyuncu sayısını 1 ile 10 arasında kendisi seçer, oda kodunu arkadaşlarıyla paylaşır (veya herkese açık listeye koyar) ve herkes aynı yeraltı tüneline, aynı anda, yan yana kazarak ilerler.

"1 kişi" seçimi teknik olarak host'un tek başına, ama çok oyunculu altyapı üzerinden (buluta bağlı) oynamasıdır — bu isterseniz gizlenebilir ve arayüzde sadece 2-10 aralığı gösterilebilir; tasarım her iki opsiyonu da destekler (bkz. 5.2).

2.2 Neden Bu Tasarım?

- Mevcut oyun döngüsüyle doğal uyum: Derin Kazı zaten "birlikte ilerlenen bir tünel" fikri üzerine kurulu; aynı ızgarada birden fazla oyuncunun kazması, oyunun temasına organik olarak oturuyor (rastgele bir "multiplayer moda" eklemek yerine).
- Retention ve viral büyüme: Arkadaşlarla oynanan oturumlar, tekil mobil oyunlara göre çok daha yüksek geri dönüş oranına sahiptir; oda kodu paylaşımı organik davet mekanizması sağlar.
- Türkiye pazarına uygunluk: Arkadaş/aile gruplarıyla oynanan "davetle katıl" modelleri (WhatsApp grup linki gibi paylaşılan oda kodları), Türkiye'deki mobil oyuncu davranışıyla örtüşüyor.
- Aşamalı risk: Solo mod hiç bozulmadan çalışmaya devam ettiği için, çok oyunculu katman başarısız olsa/ertelense bile ana ürün risk altında kalmaz.

2.3 Kapsam Dışı Bırakılanlar (Bilinçli Sınırlar)

Uzman tasarımın bir parçası da neyin şimdilik yapılmayacağına karar vermektir:

- Gerçek zamanlı sesli/görüntülü sohbet — üçüncü parti maliyeti yüksek, MVP kapsamı dışında; yerine emoji/hazır mesaj sistemi önerilir (bkz. 5.7).
- Cihazlar arası aynı anda çizim / fizik senkronizasyonu — oyun tur bazlı "hamle" mantığında kaldığı için (her hamle = 1 hücre hareket + opsiyonel 1 vuruş), tam gerçek zamanlı fizik motoru senkronizasyonuna gerek yok; bu da mimariyi önemli ölçüde basitleştiriyor.
- Küresel sıralama / ligler — ayrı bir özellik olarak gelecekte eklenebilir, bu dokümanın kapsamı dışında.

3. Oyun Tasarımı — Ekip Kazısı Modu

3.1 Oyun Modları

Mod	Oyuncu Sayısı	Açıklama
Solo Kazı	1 (mevcut)	Bugünkü çevrimdışı deneyim; değişmeden korunur, internet gerektirmez.
Ekip Kazısı — Özel Oda	Host'un seçimiyle 2-10	Oda kodu ile davetli arkadaş grubu; host oyuncu sayısı tavanını belirler.
Ekip Kazısı — Hızlı Eşleşme (Faz 2)	Sistem tarafından 2-10 arası hedeflenir	Açık odalara otomatik eşleştirme; MVP sonrası eklenir (bkz. Bölüm 9).

3.2 Oda / Lobi Sistemi

Host, ana menüden "Ekip Kazısı Oluştur" seçeneğine girer. Karşısına çıkan ekranda:

- Oyuncu sayısı seçici: 1-10 arası bir kaydırıcı (slider) veya +/- sayaç. Varsayılan değer 4 önerilir (arkadaş grubu ortalaması). Seçilen sayı, odanın *maxPlayers* alanına yazılır ve oda dolana kadar host dışındaki oyuncular tarafından değiştirilemez.
- Oda görünürlüğü: "Sadece davetle" (varsayılan, MVP) / "Herkes açık" (Faz 2, hızlı eşleşme listesinde görünür).
- Oda kodu: 6 karakterli, kolay okunur (karışabilecek 0/O, 1/I gibi karakterler hariç tutulmuş) bir kod otomatik üretilir; "Kopyala" ve "Paylaş" (WhatsApp vb. paylaşım sayfası) butonlarıyla sunulur.
- Zorluk/harita önizlemesi: Host'un o anki en yüksek derinliği baz alınarak başlangıç aşaması/derinliği gösterilir (bkz. 3.9).

Diğer oyuncular "Ekip Kazısı'na Katıl" ekranından kodu girerek lobiye katılır. Lobi ekranında: katılan herkesin adı/rengi/avatari, "Hazır" durumu, ve host için tek aktif "Kazıya Başla" butonu bulunur. Host, oda dolmasa bile (örn. 10 kişilik odada 4 kişiyle) istediği an başlatabilir — sistem oyuncu sayısını bir tavan olarak ele alır, zorunlu minimum olarak değil (host + en az 1 kişi daha katıldıysa "başlat" aktif olur; host tek başına da başlatabilir, bu durumda oturum otomatik Solo-üzeri- buluta-bağlı bir kazı olur).

3.3 Paylaşımlı Harita ve Ortak Hasar Havuzu

Ekip Kazısı modunda tüm oyuncular aynı 13×23 ızgarada bulunur (oyuncu sayısı arttıkça ölçekleme için bkz. 3.9). Her taşın (tile) can puanı (HP) ortaktır — yani bir taşın etrafında birden fazla oyuncu varsa, hepsi o taşa aynı anda vurabilir ve hasarlar toplanır.

Bu tasarım tercihinin nedeni: geleneksel "son darbeyi kim vurduysa ödülü o alır" modelinin gruplarda adaletsizlik ve tartışma yarattığı, mobil oyunlarda iyi bilinen bir sorundur ("kill stealing"). Bunun yerine hasar katkısı oranına göre ödül paylaşımı öneriyoruz:

Kural: Katkı Oranlı Ödül Paylaşımı

Bir taş kırıldığında, ödül (altın/gem/enerji) o taşa en az bir vuruş yapmış olan her oyuncu arasında, her oyuncunun verdiği toplam hasarın taşın toplam HP'sine oranına göre paylaştırılır. Örnek: 30 HP'lik bir taşa oyuncu A 20 hasar, oyuncu B 10 hasar verdiyse; ödülün %66,7'si A'ya, %33,3'ü B'ye gider (küsurahtar altın biriminde yukarı yuvarlanır, oyun ekonomisi lehine değil oyuncu lehine).

Bu mekanik, sunucu tarafında küçük bir "hasar defteri" (*contribution log*) tutmayı gerektirir — teknik detay Bölüm 6.6'da.

Hücre işgali (collision) kararı

Birden fazla oyuncunun boş/kazılmış aynı hücrede bulunmasına izin verilir (oyuncular birbirini "blokamaz"). Bu, mobil dokunmatik kontrolde hareket kilitlenmesi/sürtünme hissi yaratmamak için bilinçli bir sadeleştirmedir; görsel olarak aynı karede birden fazla oyuncu ikonu küçük rozetler halinde üst üste/yan yana gösterilir (bkz. 3.6 ve Bölüm 6.8).

3.4 Ekonomi: Kişisel Kazanç + Takım Bonusu

Ekip Kazısı'nda iki katmanlı bir ödül modeli önerilir:

Katman	Kime ait	Nasıl hesaplanır
Kişisel Kazanç	Her oyuncunun kendi cüzdanı (altın/gem/bakır)	Katkı oranlı paylaşım (3.3) ile, mevcut ödül formülleri (rewardGold vb.) aynen korunur.
Takım Tamamlama Bonusu	Aşamayı bitiren aktif tüm oyunculara, kişisel kazançları üzerinden yüzde bonus	$\%5 \times (\text{aktif oyuncu sayısı} - 1)$, 8+ oyuncuda $\%35$ ile sınırlanır (tavan). Yani 2 kişi = $\%5$, 4 kişi = $\%15$, 8+ kişi = $\%35$.
Hız Bonusu (opsiyonel)	Aşama, hedef süreden önce bitirilirse tüm takıma eşit gem	Hedef süre, oyuncu sayısına göre otomatik ölçeklenir (bkz. 3.9).

Bu model kasıtlı olarak kişisel yükseltme ekonomisini bozmuyor: Kazma/Çekiç/Şans/Enerji yükseltmeleri hâlâ tamamen kişiye özel kalıyor (mevcut *upgrade_model.dart* değişmeden kullanılabilir), sadece grup halinde oynamanın ek bir getirisi oluyor. Bu da "arkadaşını davet et, ikiniz de daha hızlı ilerleyin" mesajını net şekilde veriyor — büyümeyi teşvik eden doğru teşvik yapısı budur.

3.5 Enerji ve İşbirliği Mekanikleri

- Her oyuncunun enerjisi bireyseldir ve mevcut 25 saniyede +1 yenilenme kuralıyla aynı şekilde işler.
- "Enerji Ver" özelliği: Bir oyuncu, aşama başına 1 kez, 10 enerjisini bir takım arkadaşına gönderebilir (30 saniyelik bir bekleme/onay penceresiyle, kötüye kullanımı önlemek için). Bu, enerjisi biten ama devam etmek isteyen oyuncular için sosyal bir "kurtarma" anı yaratır.
- 2x Bonus (mevcut *activateDoubleBonus*) kişisel kalır — bir oyuncu 500 altın harcayıp kendi ödülleri katlayabilir; takıma yayılmaz (ekonomik denge için).

3.6 Sosyal Katman

- Renk/Avatar kimliği: Her oyuncuya lobiye girerken otomatik, benzersiz bir renk (10 sabit paletten) ve baş harf rozeti atanır. Izgara üzerinde oyuncular bu renkle gösterilir.
- Hazır tepkiler (Faz 1): 6-8 adet önceden tanımlı emoji/kısa mesaj ("Buraya gel!", "Harika!", "Yardım!", vb.) — serbest metin sohbeti moderasyon yükü yaratmaması için MVP'de tercih edilmez.
- Serbest metin sohbeti (Faz 2+): İstenirse, temel bir kötüye kullanım filtresiyle birlikte eklenebilir.
- Aşama sonu özet ekranı: Takımın toplam kırdığı taş sayısı, en çok katkı yapan oyuncu ("Bu Aşamanın Yıldızı"), herkesin kişisel kazancı — rekabetçi değil, kutlama amaçlı bir ekran.

3.7 Oturum Yaşam Döngüsü

- 1 Lobi: Host oda kurar, oyuncu sayısı tavanını (1-10) seçer, oyuncular katılır.
- 2 Geri sayım: Host "Başlat"a bastığında 3 saniyelik geri sayım; bu sırada geç katılanlar hâlâ girebilir (tavan dolmadıysa).
- 3 Aktif Kazı: Oyuncular ortak ızgarada hareket eder/kazar; ilerleme gerçek zamanlı senkronize olur.
- 4 Aşama Tamamlandı: Tavan/tabak HP'ler tükenince (mevcut *_advanceStage* mantığına benzer şekilde) özet ekranı gösterilir, ardından yeni aşama otomatik başlar (host onayına gerek yok — akışı kesmemek için).
- 5 Oturumu Bitir: Herhangi bir oyuncu istediği an "Odadan Ayrıl"a basabilir; bu, o oyuncuyu odadan çıkarır ama takımı durdurmaz (bkz. 3.8). Host ayrılırsa host rolü otomatik devredilir.

3.8 Bağlantı Kopması, Ayrılma ve Host Devri

Durum	Davranış
Oyuncu bağlantısı geçici kopar (ağ sorunu)	60 saniyeye kadar "Bağlanıyor..." rozetiyle grid üzerinde donuk gösterilir; süre dolmadan dönerse kaldığı yerden devam eder.
Oyuncu 60 sn içinde dönmezse	Otomatik olarak odadan çıkarılır; o ana kadarki kişisel kazancı kendisine yazılmış olarak kalır (kayıp yok), takımın geri kalanı kesintisiz devam eder.
Host ayrılır / bağlantısı tamamen kopar	Host rolü, odada en uzun süredir bulunan aktif oyuncuya otomatik devredilir ("host migration"); oturum kesilmez.
Tüm oyuncular ayrılır	Oda sunucu tarafında 5 dakika sonra otomatik temizlenir (bkz. 6.7 güvenlik/temizlik kuralları).

3.9 Oyuncu Sayısına Göre Zorluk Ölçekleme

Sabit 13×23 ızgara + sabit taş HP'leri kullanılırsa, 8-10 kişilik bir grup haritayı saniyeler içinde tüketir ve deneyim sıfır kalır. Bunu önlemek için ölçekleme kuralı:

Oyuncu Sayısı	Harita	Taş HP Çarpanı	Hedef Süre (hız bonusu)
1 (Solo-bulut)	13×23 (standart)	×1,00	—
2-3	13×23 (standart)	×1,15	Baz süre × 1,0
4-5	13×23 (standart)	×1,35	Baz süre × 1,15
6-7	17×27 (genişletilmiş)	×1,55	Baz süre × 1,3
8-10	17×31 (geniş)	×1,75	Baz süre × 1,5

Bu tablo *GridGenerator* içine parametrik olarak eklenecek katsayılardır (bkz. 6.9); mevcut üretim algoritmasının olasılık dağılımı (kaya/altın/toprak/ıksir/özel eşya oranları) değişmeden, sadece boyut ve HP çarpanı parametre olarak genişler.

4. Kullanıcı Akışı ve Yeni Ekranlar

Mevcut tek ekranlı yapıya (*MiningScreen*) ek olarak aşağıdaki ekranlar gerekiyor. Sırayla akış:

#	Ekran	Amaç	Mevcut bileşenlerle ilişki
1	Ana Menü	"Solo Kazı" / "Ekip Kazısı" seçimi	Yeni — bugün doğrudan <i>MiningScreen</i> açılıyor, önüne eklenir.
2	Ekip Kazısı Oluştur	Oyuncu sayısı seçici (1-10), görünürlük, oda kodu üretimi	Yeni ekran.
3	Ekip Kazısı'na Katıl	Kod girişi veya açık oda listesi (Faz 2)	Yeni ekran.
4	Lobi (Bekleme Odası)	Oyuncu listesi, hazır durumları, host için Başlat butonu	Yeni ekran; HudBar'daki rozet stiliyle görsel tutarlılık.
5	Kazı Ekranı (Ekip)	Mevcut <i>MiningScreen</i> 'in çoklu-oyuncu versiyonu	<i>MiningGridView</i> + <i>TileGridPainter</i> genişletilir; <i>DirectionalPad</i> , HudBar, <i>ShopPanel</i> neredeyse aynen kullanılır.
6	Aşama Özeti (Ekip)	Takım istatistikleri, MVP, kişisel kazanç dökümü	Yeni modal — <i>ShopPanel</i> 'in modal-gösterim desenine benzer şekilde kurulabilir.

Tasarım ilkesi: yeni ekranlar, mevcut *AppColors* / *AppTheme* paletini (lacivert zemin, turuncu toprak tonları, neon yeşil vurgular) birebir kullanır; oyuncu için "başka bir oyuna geçti" hissi değil, "aynı oyunun yeni bir modu" hissi yaratılır.

5. Teknik Mimari

5.1 Backend Seçimi

Dört olası yaklaşım karşılaştırıldı:

Seçenek	Artıları	Eksileri	Öneri
Firestore Realtime Database + Cloud Functions	Flutter ile birebir entegrasyon (resmî SDK); gerçek zamanlı senkron için doğal veri modeli; sunucu işletmeye gerek yok; tek geliştirici için düşük operasyon yükü; cömert ücretsiz kota.	Büyük ölçekte maliyet artabilir; gelişmiş oyun-sunucusu mantığı (tick-based simülasyon) için ideal değil.	★ MVP için önerilen seçim
Cloud Firestore	Güçlü sorgulama, iyi güvenlik kuralları, herkese açık oda listesi (Faz 2) için uygun.	Gerçek zamanlı yüksek frekanslı güncellemeler (hasar olayları) için RTDB kadar hafif değil; belge bazlı yazım maliyeti daha yüksek.	Oda listesi/matchmaking meta verisi için RTDB'ye ek olarak kullanılabilir
Supabase Realtime (Postgres tabanlı)	Açık kaynak, SQL esnekliği, iyi fiyat/performans.	Flutter ekosisteminde Firestore kadar olgun değil; solo geliştirici için öğrenme eğrisi.	Alternatif — vendor bağımlılığından kaçınmak istenirse
Özel oyun sunucusu (ör. Colyseus/Node.js, WebSocket)	Tam kontrol, en iyi hile önleme, düşük gecikme (odalı oturum modeli).	Sunucu barındırma + DevOps yükü; solo geliştirici için MVP aşamasında orantısız maliyet/karmaşıklık.	Oyuncu tabanı büyür ve rekabetçi modlar eklenirse Faz 3 seçeneği

Karar

Öneri: MVP için Firestore Realtime Database (oda/oyuncu/ızgara durumu senkronu) + Cloud Functions (yetkili hasar/ödül doğrulama) + Firestore Authentication (anonim oturum açma — kullanıcıya hesap açtırmadan, cihaz bazlı kimlik). Bu yığın, mevcut tek-geliştirici / tek-repo yapısına en az operasyonel yük ekleyen, Flutter ile en hızlı entegre olan seçenektir. Oyuncu tabanı büyüyüp rekabetçi/eşleştirmeli modlar gündeme gelirse Bölüm 9'da tarif edilen Faz 3'te özel bir oyun sunucusuna geçiş değerlendirilebilir.

5.2 Veri Modeli (Realtime Database Şeması)

Önerilen ağaç yapısı (JSON benzeri, RTDB'nin doğal formatı):

Şema Taslağı

```
rooms/{roomId}/
  meta: { hostUid, maxPlayers (1-10), visibility, status: lobby|active|finished, createdAt, stage,
  depth, gridSeed, biomeName }
  players/{uid}: { displayName, colorIndex, avatarSeed, ready, connected, position:{row,col},
  energy, maxEnergy, personalGold, personalGems, lastSeenAt }
  tiles/{tileId}: { currentHp, isCleared, lastMinerUid } — sadece dinamik alanlar; taş tipi/ödül
  tabloları seed'den yerel türetilir
  damageEvents/{eventId}: { tileId, uid, amount, ts } — Cloud Function tarafından işlenip
  temizlenen kısa ömürlü kuyruk
```

chatEmotes/{eventId}: { uid, emoteld, ts }

Önemli tasarım kararı: Izgaranın statik kısmı (hangi hücrede hangi taş tipi, başlangıç HP'si, ödül miktarları) ağ üzerinden hiç taşınmaz. Bunun yerine sadece bir *gridSeed* (tek bir sayı) senkronize edilir; her istemci, mevcut *GridGenerator*'ın seed alacak şekilde küçük bir değişiklikle (bkz. 6.9) bu seed'i kullanarak aynı haritayı yerel olarak üretir. Ağdan sadece değişen veriler (taş HP'si, kim kazdı, oyuncu pozisyonları) akar. Bu, $13 \times 23 = 299$ karelik tüm ızgarayı her güncellemede göndermek yerine bant genişliğini onlarca kat azaltır ve Firebase maliyetini doğrudan düşürür (bkz. Bölüm 7).

5.3 Gerçek Zamanlı Senkronizasyon Akışı

Bir oyuncunun kazı hamlesi, uçtan uca şu adımlardan geçer:

- 1 İstemci, yön tuşuna basıldığında iyimser (optimistic) güncelleme yapar: hedef taşın HP'sini yerel olarak azaltır, animasyonu hemen oynatır (gecikme hissettirmemek için).
- 2 Aynı anda, istemci *damageEvents* kuyruğuna { tileId, uid, amount, ts } yazar.
- 3 Bir Cloud Function (RTDB "onWrite" tetikleyicisiyle) bu olayı yakalar; *tiles/{tileId}/currentHp* üzerinde bir transaction (atomik işlem) çalıştırarak HP'yi güvenli şekilde azaltır — bu adım, iki oyuncunun aynı anda aynı taş vurmamasından doğacak yarış durumunu (race condition) çözer.
- 4 HP sıfıra inerse, Cloud Function ilgili taş a ait tüm *damageEvents* kayıtlarını toplar, katkı oranını hesaplar (3.3), her oyuncunun *personalGold/personalGems* alanlarını günceller, taş *isCleared: true* yapar ve işlenen olayları kuyruktan siler.
- 5 Tüm istemciler, dinledikleri *tiles/* ve *players/* yoluna gelen güncellemeyle uzlaşır (reconcile): kendi iyimser tahminleri sunucu sonucuyla farklıysa sessizce düzeltilir (oyuncu bunu fark etmez, çünkü fark genelde birkaç HP puanı kadardır).

Bu akış, hem düşük algılanan gecikme (iyimser güncelleme sayesinde anında tepki) hem de sunucu-yetkili doğruluk (gerçek ödül sadece Cloud Function onayından sonra kalıcılaşır) sağlar — mobil oyunlarda yaygın kullanılan bir dengedir.

5.4 Flutter Tarafında Mimari Genişleme

Mevcut kodun bozulmaması için Repository / Strateji deseni önerilir. Bugün *GameNotifier*, tüm mantığı doğrudan içinde barındırıyor. Yeni yapıda:

- *MiningRepository* adında soyut bir arayüz tanımlanır: *moveOrDig()*, *purchaseUpgrade()* gibi eylemleri ve bir *Stream<GameState>* yayınını tanımlar.
- *LocalMiningRepository*: bugünkü *GameNotifier* mantığının neredeyse birebir taşınmış hâli — Solo Kazı modu bunu kullanmaya devam eder, davranışı hiç değişmez.
- *RemoteMiningRepository*: Firebase RTDB dinleyicileriyle çalışan yeni implementasyon; iyimser güncelleme + sunucu uzlaşımı mantığını içerir (5.3).
- *MultiplayerGameNotifier*, seçilen *MiningRepository*'i enjekte alır (Riverpod *Provider.family* veya basit bir override); *GameState* modeli, diğer oyuncuların pozisyon/renk bilgisini tutan bir *List<RemotePlayerModel>* *otherPlayers* alanıyla genişletilir.

Bu sayede *mining_screen.dart*, *mining_grid_view.dart*, *tile_gridPainter.dart*, *directional_pad.dart*, *hud_bar.dart* ve *shop_panel.dart* dosyaları aynı *GameState* arayüzünü tükettikleri sürece, altta Solo mu Ekip Kazısı mı çalıştığını bilmelerine gerek kalmaz — sadece *TileGridPainter*'a diğer oyuncuları çizecek küçük bir ek mantık (bkz. 5.8) eklenir.

5.5 Yeni ve Değişecek Dosyalar (Plan — Kod Değil)

Dosya / Modül	Durum	Değişikliğin Niteliği
lib/features/mining/domain/services/grid_generator.dart	Değişecek	Rastgele üretici, sabit bir seed parametresi alacak şekilde güncellenir (deterministik hâle getirilir); boyut/HP çarpanı parametreleri eklenir (3.9).
lib/features/mining/domain/models/grid_model.dart	Değişecek	"playerPosition" tekil alanı yerine (Solo için) veya yanında (Ekip için) çoklu oyuncu pozisyon haritası taşıyacak destek eklenir.
lib/features/mining/domain/models/tile_model.dart	Küçük ekleme	"lastMinerUid" alanı eklenir (katkı takibi için); JSON metodları zaten mevcut, minimal değişir.
lib/features/mining/application/game_notifier.dart	Yeniden düzenlenir	İş mantığının bir kısmı yeni MiningRepository arayüzüne taşınır; Solo davranış korunur (LocalMiningRepository).
lib/features/multiplayer/ (yeni modül)	Yeni	domain/ (RoomModel, RemotePlayerModel), application/ (RemoteMiningRepository, LobbyNotifier), presentation/ (yeni ekranlar: menü, oda kur, oda katıl, lobi, aşama özeti).
lib/features/mining/presentation/painters/tile_gridPainter.dart	Genişletilir	Diğer oyuncuların ikon/reng/rozetlerini izgara üzerinde çizecek ek çizim mantığı.
lib/features/mining/presentation/widgets/hud_bar.dart	Küçük ekleme	Ekip Kazısı'ndayken oda kodu / oyuncu sayısı rozeti gösterecek koşullu bir alan.
lib/main.dart	Küçük ekleme	Firebase başlatma (initializeApp) ve yeni ana menü rotası eklenir.
Cloud Functions projesi (yeni, ayrı repo/klasör)	Yeni	Node.js/TypeScript: hasar doğrulama, ödül dağıtımı, oda temizliği (bkz. 5.3, 5.7).

5.6 Ekranlar Arası Durum Yönetimi

Riverpod'un *Provider.family* veya basit *ProviderScope override* mekanizmasıyla, kullanıcı "Solo Kazı"yı mı yoksa belirli bir *roomId* ile "Ekip Kazısı"nı mı oynadığı, uygulama genelinde tek bir noktadan (basit bir *AppModeProvider*) yönetilir. Bu, ekranlar arası geçişlerde tutarlılığı garanti eder ve gelecekte üçüncü bir mod eklenmesini (örn. rekabetçi mod) kolaylaştırır.

5.7 Güvenlik Kuralları ve Oda Temizliği

- Firebase Authentication (anonim) ile her cihaza benzersiz bir *uid* atanır; kullanıcıdan e-posta/şifre istenmez.
- RTDB güvenlik kuralları: bir oyuncu yalnızca kendi *players/{uid}* alt düğümüne yazabilir; *tiles/* ve ödül alanlarına doğrudan istemci yazımı kapatılır — sadece Cloud Functions (yönetici yetkisiyle) yazabilir. Bu, istemci tarafından sahte altın/gem yazılmasını mimari düzeyde imkânsız kılar.
- Zamanlanmış bir Cloud Function (ör. her 10 dakikada bir), *status: finished* olan veya 30 dakikadan uzun süredir etkin oyuncusu olmayan odaları otomatik siler — veri birikimini ve dolayısıyla depolama maliyetini kontrol altında tutar.

5.8 Görsel/UI Değişiklikleri Özeti

- Izgara üzerinde her uzak oyuncu, kendi rengiyle küçük bir daire + baş harfi olarak gösterilir; oyuncunun kendi karakteri mevcut haliyle (turuncu/mavi işçi ikonu) kalır.
- Bir taş şu an başka biri tarafından "aktif kazılıyor" ise, o taşın kenarlığı hafif parlayan bir renkle vurgulanır (kimin kazdığı bilgisini paylaşımlı hissettirmek için).
- HUD çubuğuna, sadece Ekip Kazısı'ndayken görünen küçük bir "oda kodu + oyuncu sayısı" rozeti eklenir (mevcut "x2 Bonus" rozetinin yanına, aynı görsel dilde).

6. Maliyet ve Ölçeklenebilirlik Tahmini

Firebase Realtime Database, Spark (ücretsiz) planında 1 GB depolama ve ayda 10 GB indirme (download) trafiği sunuyor; Blaze (kullandıkça öde) planına geçildiğinde bu ücretsiz kotalar korunur ve aşımında depolama için GB başına 5 USD, indirme trafiği için GB başına 1 USD ücretlendirilir; eşzamanlı bağlantı sınırı Spark'ta 100, Blaze'de veritabanı başına 200.000'dir. (Güncel ve resmî rakamlar için lansman öncesi firebase.google.com/pricing kontrol edilmelidir — fiyatlandırma zamanla değişebilir.)

Bölüm 5.2'de açıklanan "sadece seed + delta senkronu" tasarımı sayesinde, tek bir Ekip Kazısı oturumunun trafiği düşük kalır: sabit ızgara verisi hiç taşınmaz, sadece taş HP değişimleri ve oyuncu pozisyonları (her biri birkaç yüz bayt) akar. Bu, ücretsiz kotanın erken aşamada (binlerce aktif kullanıcıya kadar) yeterli olmasını beklenebilir kılar; Cloud Functions çağrıları da benzer şekilde cömert bir ücretsiz kotaya sahiptir.

Maliyeti düşük tutan tasarım kararları

Sabit harita verisinin ağdan hiç geçmemesi, sadece bir 'seed' sayısının senkronize edilmesi.

Hasar olaylarının kısa ömürlü bir kuyrukta tutulup işlendikten hemen sonra silinmesi (depolama birikmiyor).

Bitmiş/terk edilmiş odaların otomatik, zamanlanmış temizliği.

İyimser istemci güncellemeleri sayesinde her tek hamlede sunucuya round-trip beklenmemesi (algılanan performans yüksek kalırken gerçek ağ trafiği azalıyor).

Öneri: Blaze plana geçiş yapıp bütçe uyarıları (budget alerts) Firebase konsolunda kurulmalı; böylece kullanıcı sayısı beklenmedik şekilde artarsa maliyet sürprizi yaşanmaz.

7. Hile Önleme ve Adil Oyun

Risk	Önlem
İstemci sahte altın/gem/enerji değeri gönderir	Ödül alanlarına doğrudan istemci yazımı güvenlik kurallarıyla tamamen kapatılır; yalnızca Cloud Functions (sunucu yetkisiyle) yazabilir (bkz. 5.7).
Bir oyuncu anormal hızda çok fazla "hasar olayı" gönderir (hız hilesi / bot)	Cloud Function tarafında, oyuncu başına saniyede makul bir hamle sınırı (rate limit) uygulanır; sınır aşan istemcinin fazladan olayları yok sayılır.
Bir oyuncu, hiç katkı vermeden aynı odada durup takım bonusundan (3.4) faydalanmaya çalışır ("AFK sömürüsü")	Takım Tamamlama Bonusu, kişinin kendi kişisel kazancı üzerinden hesaplanır (3.4); hiç kazı yapmayan oyuncunun kişisel kazancı zaten sıfıra yakın olacağından bonus da anlamsızlaşır — ekonomik olarak kendi kendini dengeleyen bir tasarım.
Sahte/otomatik oda açıp kapatarak sunucu kaynağı tüketme	Anonim kimlik başına dakikada oda oluşturma sınırı; zamanlanmış temizlik (5.7).
İstemcide değiştirilmiş uygulama ile hileli istek gönderme	Firebase App Check entegrasyonu (isteğin gerçek, değiştirilmemiş bir uygulamadan geldiğini doğrulayan ek katman) Faz 2'de değerlendirilebilir.

8. Monetizasyon Fırsatları

Çok oyunculu katman, mevcut ekonomiyi bozmadan yeni, doğal gelir noktaları açar:

- "Arkadaş Davet Et" ödüllü reklam: Oda kodunu paylaşan host'a, biri katıldığında ödüllü reklam (rewarded video) sonrası bonus enerji/altın verilir — organik daveti teşvik ederken reklam geliri yaratır.
- Oda kozmetikleri (IAP): Özel oyuncu renkleri/avatar çerçeveleri, oda için özel "tema" (görsel filtre) — düşük geliştirme maliyetli, yüksek marjlı kozmetik satın alımlar.
- "Ekstra Enerji Paketi — Takım" IAP: Tüm takıma birden enerji hediye eden bir grup paketi; mevcut tekil enerji/altın IAP mantığının doğal bir uzantısı.
- Sezonluk Ekip Etkinlikleri: Sınırlı süreli, sadece Ekip Kazısı'nda açılan özel biome/ödülleri — geri dönüşü teşvik eden bir canlı-operasyon (live-ops) fırsatı.

9. Uygulama Yol Haritası

Aşağıdaki fazlandırma, riski aşamalı olarak yönetecek ve her fazın sonunda test edilebilir/yayınlanabilir bir sonuç verecek şekilde kurgulanmıştır. Süre tahminleri, mevcut kod tabanına aşına, tek geliştiricilik bir çalışma temposu varsayar; kesin planlama için sprint bazlı yeniden değerlendirme önerilir.

Faz 1 — Temel (MVP): Davetli Ekip Kazısı

- Firebase projesi kurulumu (Auth anonim, RTDB, Cloud Functions iskeleti).
- Deterministik (seed'li) ızgara üretimine geçiş — hem Solo hem Ekip modunda kullanılacak ortak altyapı.
- MiningRepository soyutlaması + LocalMiningRepository'ye taşınan mevcut mantık (Solo modun hiç bozulmadığının doğrulanması).
- RemoteMiningRepository: oda kur/katıl, lobi, gerçek zamanlı ızgara senkronu, katkı oranlı ödül dağıtımı (3.3–3.4).
- Yeni ekranlar: Ana Menü, Oda Kur (1–10 seçici), Oda Katıl, Lobi, Aşama Özeti.
- Temel hile önleme (5.7, 7): sunucu-yetkili ödül yazımı, hız sınırlama.

Faz 2 — Genişletme

- Hızlı eşleşme / herkese açık oda listesi.
- Enerji hediye etme, hazır tepkiler (emoji) sosyal katmanı.
- Oyuncu sayısına göre harita/HP ölçekleme (3.9) — genişletilmiş 17×27 ve 17×31 haritalar.
- Takım Tamamlama ve Hız bonuslarının canlıya alınması, dengeleme (balancing) verisiyle ince ayar.
- Firebase App Check entegrasyonu.

Faz 3 — Olgunlaşma (talebe bağlı)

- Serbest metin sohbeti (moderasyon filtresiyle).
- Sezonluk ekip etkinlikleri ve özel kozmetikler (Bölüm 8).
- Oyuncu tabanı ve eşzamanlı oturum sayısı önemli ölçüde büyürse, kritik gerçek-zamanlı yollar için özel oyun sunucusuna (Colyseus vb.) kademeli geçiş değerlendirmesi.
- Rekabetçi/yarışmalı alternatif mod ("kim daha çok altın toplar") — bu doküman kapsamı dışında, ayrı bir tasarım dokümanı gerektirir.

10. Riskler ve Azaltım Stratejileri

Risk	Olasılık/Etki	Azaltım
Gerçek zamanlı senkron gecikmesi kötü oyun hissi yaratır	Orta / Yüksek	İyimser istemci güncellemeleri (5.3) ile algılanan gecikme neredeyse sıfırlanır; RTDB'nin düşük gecikmeli mimarisi bu senaryoya uygundur.
Büyük gruplar (8-10 kişi) haritayı çok hızlı tüketip deneyimi sığlaştırır	Orta / Orta	Oyuncu sayısına göre harita/HP ölçekleme (3.9); Faz 2'de canlı veriyle ince ayar.
Toplulukta kötüye kullanım (rahatsız edici oda isimleri, spam davet)	Düşük / Orta	MVP'de serbest metin sohbeti yok (sadece hazır emoji); oda isimlendirmesi için basit kelime filtresi.
Firebase maliyeti beklenenden hızlı artar	Düşük / Orta	Bölüm 6'daki tasarım kararları (seed+delta senkron) trafiği düşük tutar; bütçe uyarıları erken sinyal verir.
Solo mod, yeni mimari sırasında kazara bozulur	Düşük / Yüksek	LocalMiningRepository'nin mevcut GameNotifier mantığının doğrudan taşınması ve Faz 1 sonunda regresyon testiyle doğrulanması (5.4, 9).

11. Sonuç

Derin Kazı'nın mevcut mimarisi — katmanlı yapı, ayrıştırılmış sunum bileşenleri, kısmen JSON'a hazır modeller — çok oyunculu bir genişleme için sağlam bir başlangıç noktası sunuyor. Önerilen Ekip Kazısı modu; host'un 1 ile 10 arasında serbestçe seçtiği oyuncu sayısı, paylaşımlı ızgara üzerinde katkı oranlı adil ödül paylaşımı, kişisel ekonomiyi koruyan takım bonusu ve Firebase tabanlı, düşük operasyon yüklü bir mimariyle; hem oyun tasarımı hem teknik fizibilite açısından mevcut ürünle tutarlı, uygulanabilir bir yol çiziyor.

Bu doküman onaylandıktan sonra, Faz 1 kapsamındaki modüllerden (öncelikle deterministik ızgara üretimi ve MiningRepository soyutlaması) başlanarak kod geliştirme aşamasına geçilebilir.

Ölmez Tech — Derin Kazı Çok Oyunculu Tasarım Dokümanı · Bu doküman tasarım/mimari amaçlıdır, üretim kodu içermez.